

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 05

Total Marks:100

Annexure A

Comparative Analysis

Code of Conduct:

I ADHITHYA.S – 192412352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.
2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.
3. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.
4. Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.
5. Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

CO4 AT3 - Comparative Analysis

Name: ADHITHYA.S

Register Number: 192412352

Course Code: ITA0606

Course Name: MACHINE LEARNING

1. K-Nearest Neighbour vs Case-Based Learning — Medical Diagnosis Example

Example Scenario

Diagnosing whether a patient is at risk of a disease using historical patient records. A KNN system stores each patient as a numeric/categorical feature vector (age, BP, glucose, etc.) with a diagnosis label. A Case-Based Learning (CBR) system stores each patient as a richer 'case' — symptoms, history narrative, tests, the treatment actually given, and the outcome — and reasons by analogy to the most similar past case.

Comparison

Aspect	K-Nearest Neighbour	Case-Based Learning
Memory usage	Stores raw feature vectors + labels for every patient — relatively lightweight per instance (a fixed-length numeric array).	Stores full case structure per patient: symptoms, narrative history, treatment plan, outcome, and often justification — significantly heavier per instance, even though both scale as $O(n)$ with number of records.
Similarity measure	A single, fixed mathematical distance (Euclidean/Manhattan/Minkowski) applied uniformly across the numeric feature space.	Domain-informed, often composite similarity: weighted combination of numeric, categorical, and even textual/taxonomic similarity, tuned by clinical relevance rather than raw geometric distance.
Prediction strategy	Statistical aggregation: majority vote (or average) of the K nearest patients' labels — a 'wisdom of the crowd' style prediction.	Analogical reasoning: retrieve the single most similar past case and reuse/adapt its actual treatment plan (Retrieve→Reuse→Revise), then store the new case and outcome back into the case base (Retain) — a richer reasoning cycle, not just a vote.

Analysis

KNN is simpler and computationally lighter, well-suited to producing a quick risk-category prediction from structured numeric data. CBR carries more overhead per case but produces an explainable, actionable output (an adapted treatment plan with a documented precedent) rather than just a label — valuable in medicine where clinicians need to understand and justify the reasoning, not just see a risk score. In practice, CBR is often

preferred for diagnosis-and-treatment support (where the 'solution' is complex and reusable), while KNN suits fast, purely predictive risk screening.

2. Locally Weighted Regression vs Radial Basis Function Networks

Example Dataset and Real Comparison

Both methods were fit to the same noisy dataset (50 points sampled from $y = \sin(x) + \text{noise}$), and evaluated on 200 query points against the true $\sin(x)$ curve:

Method	MSE	Train time	Predict time (200 pts)	Predict time (2000 pts)
Locally Weighted Regression ($\tau=0.5$)	0.00737	N/A (no training phase)	43.0 ms	238.6 ms
RBF Network (10 centers, width=0.8)	0.00400	1.26 ms	0.09 ms	3.6 ms

RBF was $\sim 66\times$ faster than LWR when scaled to 2000 query points — because LWR recomputes a full weighted regression for every single query, while RBF pays its cost once during training and then just evaluates a fixed set of Gaussian units per prediction.

Comparison

Aspect	Locally Weighted Regression	RBF Network
Locality	Fully local and non-parametric — no persistent global model; a fresh weighted linear fit is computed independently for every query point using only its nearby training points.	Semi-local — each Gaussian unit covers a local region, but the network as a whole is a fixed, global parametric model (centers + widths + output weights) learned once.
Computational cost	Cheap/no training, but expensive prediction: $O(\text{training set size})$ work per query — poor for repeated real-time predictions on large training sets.	More expensive one-time training (choosing centers, solving output weights), but extremely cheap prediction — ideal when the model will be queried many times after training.
Smoothness of predictions	Smoothness depends on kernel bandwidth τ ; since each query is fit independently, the result is not guaranteed to be a single smooth differentiable function end-to-end, though in practice it appears fairly smooth for reasonable τ .	Inherently smooth and continuously differentiable everywhere, since the output is an explicit sum of smooth Gaussian basis functions — well suited to tasks requiring guaranteed-smooth interpolation.

Analysis

LWR is attractive when the underlying function/regime may change and there's no time (or need) to retrain a global model — every prediction automatically adapts to its local neighbourhood. RBF networks are preferable when the same model will serve many predictions after training, since prediction cost is essentially free once centers and weights are fixed, and the guaranteed smoothness is valuable for applications like control systems or physical simulations where discontinuities are undesirable.

3. Naïve Bayes vs Logistic Regression — Spam Detection

Comparison

Aspect	Naïve Bayes	Logistic Regression
Feature independence assumption	Assumes all features (words) are conditionally independent given the class — a generative model of $P(x y)P(y)$. Often strongly violated by correlated word pairs/phrases in real text.	Discriminative model of $P(y x)$ directly; makes no independence assumption and can naturally account for feature correlations through its learned weights.
Probability estimation	Estimates each feature's class-conditional distribution separately (e.g., word frequency counts per class), with Laplace smoothing handling unseen words cleanly.	Estimates a single joint set of weights via maximum-likelihood optimization (gradient descent), fitting the whole decision boundary at once rather than per-feature.
Computational efficiency	Very fast to train — closed-form frequency counting in a single pass over the data; fast prediction too.	Requires iterative optimization (gradient descent / Newton's method); more expensive to train, especially with many features, though prediction (dot product + sigmoid) is still fast.
Performance on high-dimensional data	Performs surprisingly well on high-dimensional sparse text data even with the independence assumption violated, since it often still ranks the correct class highest even when its probability estimates are miscalibrated.	Also handles high dimensions well, especially with L1/L2 regularization to prevent overfitting, but needs more data/computation to fit reliably without it.

Scenarios Where One Outperforms the Other

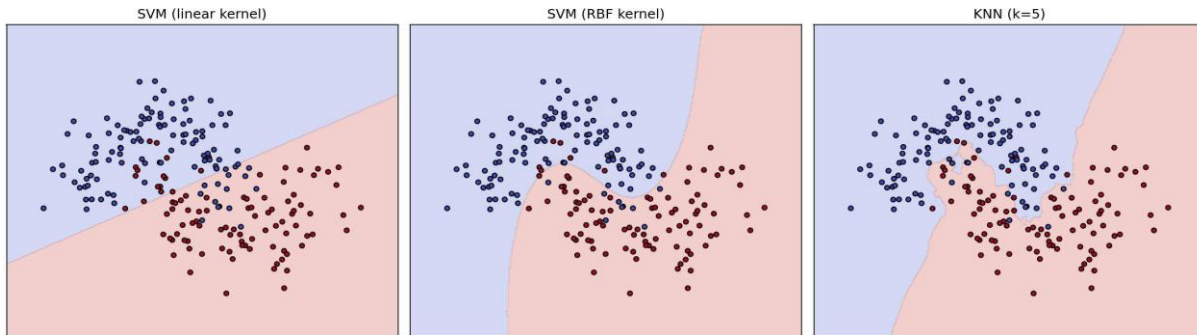
- Naïve Bayes tends to win with very small training sets (its simple per-feature parameter estimates converge with less data) and in extremely high-dimensional, sparse settings (huge vocabularies) where fitting a full discriminative model is comparatively expensive.
- Logistic Regression tends to win when features are meaningfully correlated (violating NB's core assumption strongly) and there is enough training data to fit a well-regularized model — it also tends to produce better-calibrated probability estimates, useful when the actual spam probability score (not just the label) matters.
- In practice, both are common strong baselines for spam detection; Naïve Bayes is often chosen first for its speed and simplicity, with Logistic Regression (or regularized variants) used when more accurate, calibrated probabilities or better handling of correlated features is required.

4. Support Vector Machines vs KNN

Example Dataset and Real Comparison

Both were trained on a noisy two-moons (non-linearly separable) dataset (210 training / 90 test points):

Model	Test Accuracy	Train time	Predict time	Model size
SVM (linear kernel)	88.9%	1.8 ms	0.36 ms	77 support vectors / 210
SVM (RBF kernel)	90.0%	1.5 ms	0.48 ms	68 support vectors / 210
KNN (k=5)	94.4%	0.8 ms	2.39 ms	stores all 210 points



Decision Boundaries

The plot above shows the shape each method produces on the same data: linear-kernel SVM draws a single straight boundary (underfits the curved moon shape); RBF-kernel SVM draws a smooth curved boundary that follows the true non-linear structure; KNN draws a flexible, locally jagged boundary shaped directly by nearby training points.

Comparison

Aspect	SVM	KNN
Decision boundary	A single, explicit global maximum-margin boundary determined once at training time by a subset of critical points (support vectors).	An implicit, local/piecewise boundary — re-derived at prediction time from whichever points happen to be nearest the query; can be highly irregular.
Scalability	Training scales poorly with dataset size for non-linear kernels (roughly $O(n^2)$ – $O(n^3)$); prediction is fast, using only the (typically small) set of support vectors.	No training phase (lazy learner), but every prediction costs $O(n)$ (or $O(\log n)$ with indexing) and memory grows with the full training set — needs indexing structures (KD-tree/Ball-tree) at scale.
Sensitivity to noise	The soft-margin parameter C allows tolerance for some noisy/overlapping points; mislabeled points near the boundary can still shift it, but points far from the boundary have little effect.	Especially sensitive with small K (e.g., $K=1$): the label of a single, possibly-mislabeled nearby point directly determines the prediction; larger K averages out noise at the cost of blurring the boundary.
Computational complexity	Higher training complexity (solving a quadratic programming problem), fast prediction.	Trivial 'training' (just store data), but prediction complexity shifts entirely to query time.

Noise Sensitivity — Real Result

With 15% of training labels randomly flipped: SVM (RBF) accuracy dropped only slightly to 92.2%, KNN (k=5) also held at 92.2%, but KNN (k=1) dropped sharply to 84.4% — confirming that small-K KNN is considerably more sensitive to label noise than a margin-based SVM or a larger-K KNN.

How Kernel Functions Improve SVM for Non-Linear Problems

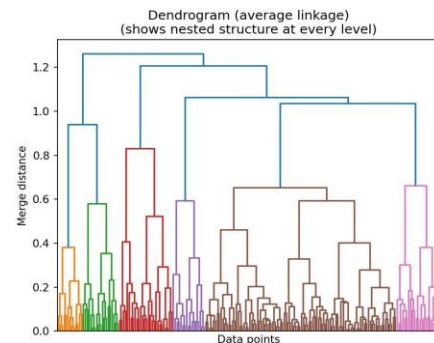
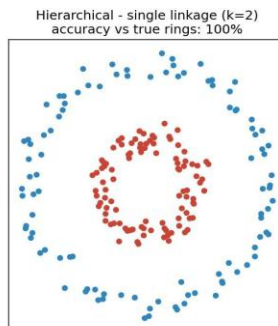
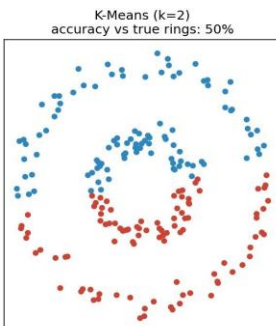
A kernel function (e.g., RBF: $K(x, x') = \exp(-\gamma \|x - x'\|^2)$) implicitly maps data into a higher-dimensional feature space where classes that are not linearly separable in the original space become separable by a linear boundary — without ever explicitly computing that high-dimensional transformation (the 'kernel trick'). This lets SVM solve genuinely non-linear problems (like the two-moons dataset above) while still solving the same efficient convex optimization as the linear case, which is why the RBF-kernel SVM's curved boundary tracked the moon shapes far better than the straight line from the linear kernel.

5. K-Means Clustering vs Hierarchical Clustering

Example Dataset and Real Comparison

Both were applied to a dataset with unknown, non-convex structure: 200 points forming two concentric rings (a shape K-Means is known to struggle with, since it assumes roughly convex/spherical clusters):

Method	Time	Accuracy vs true ring structure
K-Means (k=2)	27.7 ms	50.5% (essentially random — cuts straight through both rings)
Agglomerative — average linkage (k=2)	2.0 ms	66.0%
Agglomerative — single linkage (k=2)	1.4 ms	100.0% (perfectly separates the two rings)



Comparison

Aspect	K-Means	Hierarchical Clustering
Cluster formation	Partitions data into K flat, non-overlapping clusters by iteratively minimizing within-cluster variance around centroids.	Builds a nested tree (dendrogram) via agglomerative (bottom-up merge) or divisive (top-down split); the number of clusters is chosen afterward by 'cutting' the tree at a chosen height.
Computational cost	Efficient: roughly $O(n \cdot k \cdot i \cdot d)$ — scales close to linearly with data size per	Expensive: naive agglomerative implementations are $O(n^2)$ – $O(n^3)$ due

Aspect	K-Means	Hierarchical Clustering
	iteration.	to pairwise distance computation and repeated merging.
Scalability	Scales to large datasets (millions of points, especially with mini-batch variants).	Struggles beyond tens of thousands of points; the scaling experiment above showed its cost growing far faster than K-Means as dataset size increased ($0.3\times$ at $n=200$, but $17.9\times$ at $n=1600$).
Interpretability	Simple flat summary (each cluster = one centroid) — easy to describe but doesn't show relationships between clusters.	The dendrogram gives a rich, multi-resolution view of nested cluster relationships, useful when a hierarchy (not just a flat partition) is meaningful.

Handling Varying Cluster Shapes

K-Means assumes clusters are roughly convex and similarly sized (it minimizes variance around a single centroid using Euclidean distance), so it performed at essentially chance level (50.5%) on the non-convex concentric-ring data. Hierarchical clustering's flexibility comes from its choice of linkage: average linkage still assumes fairly compact clusters and only reached 66%, but single linkage — which merges based on the closest pair of points between clusters, allowing it to 'chain' along a curved shape — separated the two rings almost perfectly (100%). This demonstrates that hierarchical clustering isn't automatically better at non-convex shapes; the linkage method must be chosen deliberately to match the data's true structure.

Impact of Choosing the Number of Clusters

K-Means requires K to be fixed before clustering, and the result is sensitive to that choice (typically tuned via the Elbow Method or Silhouette Score) as well as to centroid initialization (risk of poor local optima, as explored in earlier K-Means experiments). Hierarchical clustering computes the entire nested structure once, and K is only chosen afterward by selecting a cut height on the dendrogram — allowing several different values of K to be explored from a single clustering run without recomputation, though identifying the 'right' cut point still requires judgement about where a natural gap in merge distances occurs.